Задание 1. Регулярные выражения и автоматы (5 баллов)

Алгоритм

1. **НКА (Томпсон):** рекурсивно строить по regex: база (символ/ε), объединение (новые старт/финиш с ε-переходами), конкатенация (соединить выход r_1 с входом r_2), звезда (цикл через ε).
2. **ДКА (Subset Construction):** старт = ε-closure({q_0}); для каждого множества S и символа a: move(S,a) → ε-closure → новое состояние ДКА.
3. **Минимизация (Хопкрофт):** разбиение {F, Q\F}; итеративно делить классы, если переходы по символу ведут в разные классы; до неподвижной точки.
4. **Тест:** прогнать строку через таблицу.

Пример: a(a|b)b над {a,b}

НКА: 0→ᵃ1→ᵃ2→ᵇ4(F); 1→ᵇ3→ᵇ4 **ДКА:**

Сост.	Множество	a	b
A	{0}	B={1}	∅
B	{1}	C={2}	D={2}
C	{2}	∅	E={3}
D	{2}	∅	E={3}
E	{3}	∅	F={4}
F	{4}	∅	∅

Минимизация: C≡D (одинаковые переходы) → объединяем в CD. Ловушка ∅ удаляется.

Минимальный ДКА:

Сост.	a	b
A (старт)	B	—
B	CD	CD
CD	—	E
E (финиш)	—	—

Тесты: принимаемые: aab, abb; отвергаемые: aa, ba.

Задание 2. Грамматика и парсинг (5 баллов)

Алгоритм

1. **Устранение левой рекурсии:** для A → Aα | β ввести A': A → βA', A' → αA' | ε.
2. **Левая факторизация:** для A → $\alpha\beta_1$ | $\alpha\beta_2$ → A → αA', A' → β_1 | β_2.
3. **FIRST:** итеративно: терминал → {терминал}; X → $Y_1 \ldots Y_k$ → FIRST(Y_1){ε} + если ε∈FIRST(Y_1), то FIRST(Y_2){ε} и т.д.
4. **FOLLOW:** итеративно: $∈FOLLOW(S); A→αBβ → FIRST(β){ε}⊆FOLLOW(B); A→αB или ε∈FIRST(β) → FOLLOW(A)⊆FOLLOW(B).
5. **Проверка LL(1):** для A→α|β: FIRST(α)∩FIRST(β)=∅; если ε∈FIRST(α), то FIRST(β)∩FOLLOW(A)=∅.
6. **Таблица M[A,a]:** для A→α и a∈FIRST(α) → M[A,a]=A→α; если ε∈FIRST(α), для b∈FOLLOW(A) → M[A,b]=A→α.

Пример: E → E+T | T, T → T*F | F, F → (E) | id

Устранение левой рекурсии:

```
E  → TE'      E' → +TE' | ε
T  → FT'      T' → *FT' | ε
F  → (E) | id
```

FIRST/FOLLOW: - FIRST(E)=FIRST(T)=FIRST(F)={(, id} - FIRST(E')={+, ε}, FIRST(T')={*, ε} - *FOLLOW(E)=FOLLOW(E')={), $}* - *FOLLOW(T)=FOLLOW(T')={+,), $}* - *FOLLOW(F)={*, +,), $}

Проверка LL(1): конфликтов нет → грамматика LL(1).

Таблица:

NT	id	+	*	(	)	$
E	E→TE'			E→TE'		
E'		E'→+TE'			E'→ε	E'→ε
T	T→FT'			T→FT'		

NT	id	+	*	(	)	$
T’		T’→ε	T’→*FT’		T’→ε	T’→ε
F	F→id			F→(E)		

Задание 3. Семантика и типы (5 баллов)

Алгоритм

1. **Стек областей:** push при входе в блок, pop при выходе.
2. **Объявления `let x[:T]=e`:** вычислить type(e); если T есть, проверить type(e)=T; иначе type(x)=type(e); проверить повтор в текущей области (затенение из внешней разрешено).
3. **Типизация:** литерал n→int; true/false→bool; x→lookup(x).type; e_1+e_2 требует int,int→int; $e_1<e_2$ требует int,int→bool; new int[n] требует int→int[]; a[i] требует int[],int→int.
4. **Присваивание x=e:** lookup(x)≠null; x mutable; type(e)=type(x).
5. **Ошибки:** использование до объявления, вне области, несовпадение типов, повтор в области.

Пример

```
let x: int = 10;
{
   let y = x + 5;
   let z: bool = x;       // Ошибка 1
   let x = "hello";       // Ошибка 2
}
let w = y;                 // Ошибка 3
```

Области: - Scope 0 (Global): x:int - Scope 1 (Block): y:int (выведен), z:bool, x:string (затенение ок)

Ошибки: 1. Строка 4: аннотация bool, но type(x)=int → несовпадение типов. 2. Строка 5: литерал “hello” не входит в типы микроязыка (int, bool, int[]). 3. Строка 7: lookup(y) в Scope 0 = null (y объявлена в Scope 1, который закрыт) → вне области.

Выведенные типы: y:int (от операции + над int).

Задание 4. IR, оптимизация и кодогенерация (5 баллов)

Алгоритм

1. **Понижение в TAC:** разложить сложные выражения на временные: $x=e_1\ \text{op}\ e_2 \rightarrow t_1=e_1;\ t_2=e_2;\ t_3=t_1\ \text{op}\ t_2;\ x=t_3$.
2. **CFG:** лидеры = первая инструкция + цели переходов + инструкции после переходов. Базовый блок = от лидера до следующего лидера/перехода. Рёбра: goto→цель; if→цель+следующий; return→EXIT; иначе→следующий.
3. **Оптимизации:**
   - **Const Prop:** x=c → заменить чтения x на c.
   - **Алгебра:** x+0→x, x*1→x, x-0→x, c_1 op $c_2 \rightarrow c_3$.
   - **DCE:** удалить x=e, если x не живо и e без побочных эффектов/не аварийно.
4. **Стековый байт-код:** x=c → iconst c; istore slot(x); x=y op z → iload y; iload z; iop; istore x. max_stack = пик высоты стека.

Пример: `x=2+3; y=x*1; z=y-0;`

Исходный TAC:

```
t1 = 2 + 3
x  = t1
t2 = x * 1
y  = t2
t3 = y - 0
z  = t3
```

CFG: один базовый блок B1 (нет ветвлений).

Оптимизации: 1. Const folding: t1=5 2. Copy Prop: x=5, t2=5*1 3. Алгебра: 5*1→5, t2=5, y=5 4. Алгебра: 5-0→5, t3=5, z=5 5. DCE: удалить t1,t2,t3 (не используются)

Оптимизированный TAC:

```
x = 5
y = 5
z = 5
```